Zaawansowane algorytmy wizyjne

Skrypt do ćwiczeń laboratoryjnych

Tomasz Kryjak, Mateusz Wąsala, Hubert Szolc, Piotr Wzorek

Copyright © 2026
Tomasz Kryjak
Mateusz Wąsala
Hubert Szolc
Piotr Wzorek
Krzysztof Błachut

Zespół Wbudowanych Systemów Wizyjnych
Laboratorium Systemów Wizyjnych
WEAIiIB, AGH w Krakowie

Wydanie czwarte, zmienione i poprawione
Kraków, luty 2026

Spis treści

I Blok 1

1 Laboratorium 1

1 Algorytmy wizyjne w Python 3.X – wstęp 9
1.1 Wykorzystywane oprogramowanie 9
1.2 Moduły Pythona wykorzystywane w przetwarzaniu obrazów 9
1.3 Operacje wejścia/wyjścia 10
1.4 Konwersje przestrzeni barw 11
1.4.1 OpenCV 11
1.4.2 Matplotlib 11
1.5 Skalowanie, zmiana rozdzielczości przy użyciu OpenCV 12
1.6 Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy 12
1.7 Wyliczenie histogramu 13
1.8 Wyrównywanie histogramu 13
1.9 Filtracja 14

2 Laboratorium 2

2 Detekcja pierwszoplanowych obiektów ruchomych 17
2.1 Wczytywanie sekwencji obrazów 17

2.2 Odejmowanie ramek i binaryzacja 18
2.3 Operacje morfologiczne 18
2.4 Indeksacja i prosta analiza 19
2.5 Ewaluacja wyników detekcji obiektów pierwszoplanowych 20
2.6 Przykładowe rezultaty algorytmu do detekcji ruchomych obiektów pierwszoplanowych 21

3 Laboratorium 3

3 Segmentacja obiektów pierwszoplanowych 25
3.1 Cel zajęć 25
3.2 Segmentacja obiektów pierwszoplanowych 25
3.3 Metody oparte o bufor próbek 26
3.4 Aproksymacja średniej i mediany (tzw. sigma-delta) 28
3.5 Polityka aktualizacji 28
3.6 OpenCV – GMM/MOG 29
3.7 OpenCV – KNN 29
3.8 Przykładowe rezultaty algorytmu do segmentacji obiektów pierwszoplanowych 29
3.9 Wykorzystanie sieci neuronowej do segmentacji obiektów pierwszoplanowych 32

4 Extra laboratorium 1

4 Uogólniona transformata Hougha 35
4.1 Cel zajęć 35
4.1.1 R-table 35
4.2 Implementacja uogólnionej transformaty Hougha 35

5 Mini projekt 1

5 Projekt 1 41
5.1 Cel projektu 42
5.2 Przykładowa literatura 42

Blok 1

1 Laboratorium 1

1 Algorytmy wizyjne w Python 3.X – wstęp 9

1.1 Wykorzystywane oprogramowanie
1.2 Moduły Pythona wykorzystywane w przetwarzaniu obrazów
1.3 Operacje wejścia/wyjścia
1.4 Konwersje przestrzeni barw
1.5 Skalowanie, zmiana rozdzielczości przy użyciu OpenCV
1.6 Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy
1.7 Wyliczenie histogramu
1.8 Wyrównywanie histogramu
1.9 Filtracja

1. Algorytmy wizyjne w Python 3.X – wstęp

W ramach ćwiczenia zaprezentowane/przypomniane zostaną podstawowe informacje związane z wykorzystaniem języka Python do realizacji operacji przetwarzania i analizy obrazów, a także sekwencji wideo.

1.1 Wykorzystywane oprogramowanie

Przed rozpoczęciem ćwiczeń **proszę utworzyć** własny katalog roboczy w miejscu podanym przez Prowadzącego. Nazwa katalogu powinna być związana z Państwa nazwiskiem – zalecana forma to `NazwiskoImie` bez polskich znaków.

Do przeprowadzania ćwiczeń można wykorzystać jedno z trzech środowisk programowania w Pythonie – Spyder5, PyCharm lub Visual Studio Code (VSCode). Ich zaletą jest możliwość łatwego podglądu tablic dwuwymiarowych (czyli obrazów). Spyder jest prostszy, ale także ma mniejsze możliwości i więcej niedociągnięć. PyCharm jest oprogramowaniem komercyjnym udostępnianym także w wersji darmowej (wygląd zbliżony do CLion dla C/C++). W PyCharmie pracę należy zacząć od utworzenia projektu, Spyder pozwala na pracę na pojedynczych plikach (bez konieczności tworzenia projektu).

Visual Studio Code jest darmowym, lekkim, intuicyjnym i łatwym w obsłudze edytorem kodu źródłowego. Jest to uniwersalne oprogramowanie do dowolnego języka programistycznego. Konfiguracja programu do ćwiczeń sprowadza się jedynie do wybrania odpowiedniego interpretera, a dokładnie odpowiedniego środowiska wirtualnego w języku Python.

1.2 Moduły Pythona wykorzystywane w przetwarzaniu obrazów

Python udostępnia wbudowane kontenery (np. listy), ale nie oferuje natywnego typu tablicowego zaprojektowanego do wydajnych operacji wektorowych/element–po–elemencie (np. dodawania, mnożenia czy progowania całych bloków danych). W praktyce standardem jest użycie modułu *NumPy*, który dostarcza typ `ndarray` oraz mechanizmy takie jak wektoryzacja i *broadcasting*. Obraz cyfrowy można wtedy traktować jako tablicę $H \times W$ (skala

szarości) lub $H \times W \times C$ (kolor), zwykle o typie `uint8` (zakres 0–255) albo `float` (np. 0–1). *NumPy* stanowi fundament dla większości bibliotek używanych dalej do przetwarzania i wizualizacji obrazów.

Najczęściej spotkane pakiety wspierające przetwarzanie obrazów to m.in.:

- *SciPy* (moduł *ndimage*) – podstawowe operacje filtracji i przekształceń (również wielowymiarowych) oraz *Matplotlib* (*pyplot*) – wizualizacja obrazów i wykresów,
- *Pillow* (fork starszego, nierozwijanego modułu PIL) – wygodne wczytywanie/zapisywanie i proste operacje na obrazach,
- *OpenCV* (moduł *cv2*) – rozbudowany zestaw algorytmów widzenia komputerowego (filtry, geometria, cechy, dopasowanie, kalibracja, itp.),
- *scikit-image* (*skimage*) – biblioteka z algorytmami przetwarzania obrazów zintegrowana z ekosystemem naukowym Pythona,
- *Plotly* (*plotly*) – interaktywne wykresy i wizualizacje (np. mapy cieplne, wykresy 3D, dashboardy w Jupyter/HTML),
- *Rerun* (*rerun*, *rerun.io*) – logowanie i interaktywna wizualizacja danych (np. obrazy, adnotacje, 3D/robotyka) w viewerze Rerun,
- *pandas* – narzędzia do pracy z danymi tabelarycznymi (np. wczytywanie adnotacji i metadanych z CSV/Excel, analiza wyników eksperymentów).

W tym kursie oprzemy się głównie na *OpenCV*. Równolegle będziemy używać *Matplotlib* (zwłaszcza do wyświetlania wyników; uwaga: *OpenCV* domyślnie przechowuje kolor w kolejności BGR, a *Matplotlib* oczekuje RGB) oraz sporadycznie *ndimage* – gdy dana operacja nie jest dostępna w *OpenCV* albo jest tam mniej wygodna w użyciu.

1.3 Operacje wejścia/wyjścia

Wczytywanie, wyświetlanie i zapisywanie obrazu z wykorzystaniem OpenCV oraz Matplotlib

Ćwiczenie 1.1 Wykonaj zadanie, w którym przećwiczysz obsługę plików z wykorzystaniem OpenCV oraz Matplotlib.

1. Ze strony kursu pobierz obraz *mandril.jpg* i umieść go we własnym katalogu roboczym.
2. Wczytaj, wyświetl oraz zapisz obraz z rozszerzeniem .png pod nową nazwą wykorzystując bibliotekę OpenCV oraz Matplotlib. Następnie umieść na obrazie **punkt** i **prostokąt** oraz wyświetl na ekranie.

R Wykorzystaj:

- OpenCV: `cv2.imread()`, `cv2.imshow()`, `cv2.imwrite()`, `cv2.waitKey()`, `cv2.destroyAllWindows()`, `cv2.rectangle()`.
- Matplotlib: `plt.imread()`, `plt.imsave()`, `plt.figure()`, `plt.imshow()`, `plt.axis()`, `plt.show()` oraz `matplotlib.patches.Rectangle()`.

R Funkcja `waitKey()` wyświetla obraz przez określony czas (argument 0 oznacza oczekiwanie na naciśnięcie klawisza). Niepotrzebne okno można zamknąć poleceniem `destroyWindow()` z odpowiednim parametrem, a wszystkie otwarte okna — poleceniem `destroyAllWindows()`. Dobrą praktyką jest zawsze stosowanie funkcji `cv2.destroyAllWindows()`.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

1.4 Konwersje przestrzeni barw

1.4.1 OpenCV

Do konwersji przestrzeni barw służy funkcja *cvtColor*.

```
IG = cv2.cvtColor(I, cv2.COLOR_BGR2GRAY)
IHSV = cv2.cvtColor(I, cv2.COLOR_BGR2HSV)
```

Uwaga! Proszę zauważyć, że w OpenCV odczyt jest w kolejności BGR, a nie RGB. Może to być istotne w przypadku ręcznego manipulowania pikselami. Pełna lista dostępnych konwersji wraz ze stosownymi wzorami w dokumentacji OpenCV

Ćwiczenie 1.2 Wykonaj konwersję przestrzeni barw podanego obrazu.

1. Dokonać konwersji obrazu *mandril.jpg* do odcieni szarości i przestrzeni HSV. Wynik wyświetlić.
2. Wyświetlić składowe H, S, V obrazu po konwersji.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Przydatna składnia:

```
IH = IHSV[:,:,0]
IS = IHSV[:,:,1]
IV = IHSV[:,:,2]
```

1.4.2 Matplotlib

Tu wybór dostępnych konwersji jest dość ograniczony.

1. RGB do odcieni szarości. Można wykorzystać rozbicie na poszczególne kanały i wzór:

$$G = 0.299 \cdot R + 0.587 \cdot G + 0.144 \cdot B \tag{1.1}$$

Całość można opakować w funkcję:

```
def rgb2gray(I):
  return 0.299*I[:,:,0] + 0.587*I[:,:,1] + 0.114*I[:,:,2]
```

Uwaga! Przy wyświetlaniu należy ustawić mapę kolorów. Inaczej obraz wyświetli się w domyślnej, która nie jest bynajmniej w odcieniach szarości: `plt.gray()`.

2. RGB do HSV.

```
import matplotlib # add at the top of the file
I_HSV = matplotlib.colors.rgb_to_hsv(I)
```

1.5 Skalowanie, zmiana rozdzielczości przy użyciu OpenCV

Ćwiczenie 1.3 Przeskaluj obraz *mandril*. Do skalowania służy funkcja `resize`.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Przykład użycia:

```
height, width = I.shape[:2] # retrieving elements 1 and 2, i.e. the corresponding
    height and width
scale = 1.75   # scale factor
Ix2 = cv2.resize(I,(int(scale*height),int(scale*width)))
cv2.imshow("Big Mandril",Ix2)
```

1.6 Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy

Obrazy są macierzami, a zatem operacje arytmetyczne są dość proste – tak jak w pakiecie Matlab. Należy oczywiście pamiętać o konwersji na odpowiedni typ danych. Zwykle dobrym wyborem będzie double.

Ćwiczenie 1.4 Wykonaj operacje arytmetyczne na obrazie *lena*.

1. Pobierz ze strony kursu obraz *lena*, a następnie go wczytaj za pomocą funkcji z OpenCV – dodaj ten fragment kodu do pliku, który zawiera wczytywanie obrazu *mandril*. Wykonaj konwersję do odcieni szarości. Dodaj macierze zawierające mandryla i Leny w skali szarości. Wyświetl wynik.
2. Podobnie wykonaj odjęcie i mnożenie obrazów.
3. Zaimplementuj kombinację liniową obrazów.
4. Ważną operacją jest moduł z różnicy obrazów. Można ją wykonać „ręcznie” – konwersja na odpowiedni typ, odjęcie, moduł (`abs`), konwersja na *uint8*. Alternatywa to wykorzystanie funkcji *absdiff* z OpenCV. Proszę obliczyć moduł z różnicy „szarych” wersji mandryla i Leny.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Uwaga! Przy wyświetleniu obraz może nie być poprawny, ponieważ jest on typu *float64* (*double*). W praktyce należy:
1. ograniczyć zakres wartości (`clip`),
2. ewentualnie przeskalować do przedziału `[0,255]`,
3. skonwertować do *uint8*.

```
img8 = (255*np.clip(img,0,1)).astype(np.uint8) # when values are in [0,1] range
img8 = np.clip(img,0,255).astype(np.uint8) # when values are in [0,255] range
```

1.7 Wyliczenie histogramu

Obliczanie histogramu można wykonać z wykorzystaniem funkcji `calcHist`. Zanim jednak do tego przejdziemy, przypomnijmy sobie podstawowe struktury sterowania w Pythonie: funkcje i podprogramy. Proszę samodzielnie dokończyć poniższą funkcję, która oblicza histogram obrazu w 256 odcieniach szarości:

```
def hist(img):
  h=np.zeros((256,1), np.float32) # creates and zeros single-column arrays
  height, width = img.shape[:2] # shape - we take the first 2 values
    for y in range(height):
...
return h
```

Histogram można wyświetlić, korzystając z funkcji `plt.hist` lub `plt.plot` z biblioteki *Matplotlib*.

Funkcja `calcHist` umożliwia obliczenie histogramu dla kilku obrazów (lub ich składowych), dlatego jako parametry otrzymuje tablice (np. obrazów), a nie pojedynczy obraz. Najczęściej jednak wykorzystywana jest następująca postać:

```
hist = cv2.calcHist([IG],[0],None,[256],[0,256])
# [IG] -- input image
# [0] -- for greyscale images there is only one channel
# None -- mask (you can count the histogram of a selected part of the image)
# [256] -- number of histogram bins
# [0, 256] -- the range over which the histogram is calculated
```

Proszę sprawdzić czy histogramy uzyskane obiema metodami są takie same.

1.8 Wyrównywanie histogramu

Wyrównywanie histogramu to popularna i ważna operacja przetwarzania wstępnego.

1. Wyrównywanie "klasyczne"jest metodą globalną — wykonuje się je na całym obrazie. W bibliotece OpenCV znajduje się gotowa funkcja realizująca ten typ wyrównania:

   ```
   IGE = cv2.equalizeHist(IG)
   ```

2. Wyrównywanie CLAHE (ang. *Contrast Limited Adaptive Histogram Equalization*) – metoda adaptacyjna, która poprawia warunki oświetleniowe na obrazie. W przeciwieństwie do klasycznego wyrównywania histogramu działa lokalnie: wyrównuje histogram w poszczególnych fragmentach obrazu, a nie dla całego obrazu. Metoda działa następująco:
   - podział obrazu na rozłączne (kwadratowe) bloki,
   - wyznaczenie histogramu w każdym bloku,

- wyrównanie histogramu z ograniczeniem maksymalnej „wysokości” (nadmiar jest redystrybuowany na sąsiednie przedziały),
- interpolacja wartości pikseli na podstawie histogramów dla sąsiednich bloków (zwykle z uwzględnieniem czterech najbliższych sąsiadów).

Szczegóły znajdują się na Wiki oraz w tutorialu OpenCV.

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
# clipLimit - maximum height of the histogram bar - values above are distributed
    among neighbours
# tileGridSize - size of a single image block (local method, operates on separate
    image blocks)
I_CLAHE = clahe.apply(IG)
```

Ćwiczenie 1.5 Uruchom i porównaj obie metody wyrównywania.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

1.9 Filtracja

Filtracja to bardzo ważna grupa operacji na obrazach. W ramach ćwiczenia proszę uruchomić:

- filtrację Gaussa (`GaussianBlur`)
- filtrację Sobela (`Sobel`)
- Laplasjan (`Laplacian`)
- medianę (`medianBlur`)

R Pomocna będzie dokumentacja OpenCV.

Proszę zwrócić uwagę również na inne dostępne funkcje:

- filtrację bilateralną,
- filtry Gabora,
- operacje morfologiczne.

Ćwiczenie 1.6 Uruchom i porównaj wymienione metody.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

2 Laboratorium 2

2 Detekcja pierwszoplanowych obiektów ruchomych 17
2.1 Wczytywanie sekwencji obrazów
2.2 Odejmowanie ramek i binaryzacja
2.3 Operacje morfologiczne
2.4 Indeksacja i prosta analiza
2.5 Ewaluacja wyników detekcji obiektów pierwszoplanowych
2.6 Przykładowe rezultaty algorytmu do detekcji ruchomych obiektów pierwszoplanowych

2. Detekcja pierwszoplanowych obiektów ruchomych

Co to są obiekty pierwszoplanowe?

To obiekty, które są dla nas (w kontekście rozważanej aplikacji) istotne. Najczęściej są to: ludzie, zwierzęta, samochody (lub inne pojazdy) oraz bagaże (potencjalne bomby). Definicja jest więc ściśle związana z docelową aplikacją.

Czy segmentacja obiektów pierwszoplanowych to segmentacja obiektów ruchomych?

Nie. Po pierwsze, obiekt, który się zatrzymał, nadal może być dla nas interesujący (np. człowiek stojący przed przejściem dla pieszych). Po drugie, istnieje wiele ruchomych elementów sceny, które nie są dla nas istotne (np. płynąca woda, fontanna, poruszające się drzewa czy krzaki). Warto też zauważyć, że w przetwarzaniu obrazu często analizuje się ruch. Dlatego detekcję obiektów pierwszoplanowych można (i warto) wspomagać detekcją obiektów ruchomych.

Najprostsza metoda detekcji obiektów ruchomych polega na odejmowaniu kolejnych (sąsiednich) ramek. W ramach ćwiczenia zrealizujemy proste odejmowanie dwóch ramek, połączone z binaryzacją, indeksacją i analizą otrzymanych obiektów. Na koniec spróbujemy potraktować wynik odejmowania jako rezultat segmentacji obiektów pierwszoplanowych i sprawdzimy jakość tej segmentacji.

2.1 Wczytywanie sekwencji obrazów

Wykorzystane sekwencje pochodzą ze zbioru danych dostępnego na stronie changedetection.net. W razie problemów z dostępem do serwisu zbiór jest udostępniony na platformie UPeL. Zbiór zawiera sekwencje z etykietami, tzn. każdej klatce obrazu przypisano referencyjną maskę obiektów (ang. *ground truth*), w której każdy piksel należy do jednej z pięciu kategorii: tło (0), cień (50), poza obszarem zainteresowania (85), nieznany ruch (170) oraz

obiekty pierwszoplanowe (255) – w nawiasach podano odpowiadające poziomy szarości.

W ramach ćwiczenia interesuje nas jedynie podział na obiekty pierwszoplanowe oraz pozostałe kategorie. Dodatkowo w folderze znajduje się maska obszaru zainteresowania (ROI) oraz plik tekstowy z przedziałem czasowym, dla którego należy analizować wyniki (`temporalROI.txt`) – szczegóły w dalszej części ćwiczenia.

Ćwiczenie 2.1 Wczytaj sekwencje. ■

2.2 Odejmowanie ramek i binaryzacja

W celu detekcji elementów ruchomych od ramki bieżącej odejmujemy ramkę poprzednią; należy zaznaczyć, że w praktyce obliczamy wartość bezwzględną tej różnicy. Następnie, aby dokonać binaryzacji, trzeba wyznaczyć próg i dobrać go tak, aby obiekty były względnie wyraźne. Proszę zwrócić uwagę na artefakty związane z kompresją.

R Aby uniknąć problemów z odejmowaniem liczb bez znaku (*uint8*), należy wykonać konwersję do typu *int* – `IG = IG.astype('int')`.

```
(T, thresh) = cv2.threshold(D,10,255,cv2.THRESH_BINARY)
# D -- input array
# 10 -- threshold value
# 255 -- maximum value to use with the THRESH_BINARY and THRESH_BINARY_INV
    thresholding types
# cv2.THRESH_BINARY -- thresholding type
# T -- our threshold value
# thresh -- output image
```

R Pierwszy argument zwracanych wartości przez funkcję `cv2.threshold` to próg binaryzacji (jest on użyteczny w przypadku stosowania automatycznego wyznaczenia progu np. metodą Otsu lub trójkątów). Szczegóły w dokumentacji OpenCV.

Ćwiczenie 2.2 Wykonaj odpowiednie operacje, aby uzyskać zbinaryzowany obraz. ■

2.3 Operacje morfologiczne

Operacje morfologiczne to proste operacje przetwarzania obrazu binarnego (czasem także w skali szarości), które modyfikują kształt obiektów na podstawie tzw. elementu strukturalnego (np. 3×3).

- erozja (*erosion*) – redukuje obiekty: zmniejsza je, usuwa drobne jasne szumy, może rozrywać cienkie połączenia,
- dylatacja (*dilation*) – rozszerza obiekty: powiększa je, domyka małe przerwy/dziury, skleja bliskie fragmenty,
- otwarcie (*opening* = erozja → dylatacja) – usuwa drobne obiekty/szum, wygładza kontur,
- domknięcie (*closing* = dylatacja → erozja) – wypełnia małe dziury i przerwy, łączy szczeliny.

Uzyskany obraz jest dość zaszumiony. Celem jest uzyskanie jak najlepiej widocznej sylwetki przy możliwie małych zakłóceniach. Dla poprawy wyniku warto dodać etap filtracji medianowej (przed operacjami morfologicznymi) oraz w razie potrzeby skorygować próg binaryzacji.

Ćwiczenie 2.3 Wykonaj filtrację, wykorzystując erozję i dylatację (`erode` i `dilate` z OpenCV). ■

2.4 Indeksacja i prosta analiza

W kolejnym etapie należy przefiltrować uzyskany wynik. W tym celu zostanie wykorzystana indeksacja (nadanie etykiet grupom połączonych pikseli) oraz obliczenie parametrów tych grup. Służy do tego funkcja `connectedComponentsWithStats`. Wywołanie:

```
retval, labels, stats, centroids =  cv2.connectedComponentsWithStats(B)
# retval --  total number of unique labels
# labels -- destination labelled image
# stats -- statistics output for each label, including the background label.
# centroids -- centroid output for each label, including the background label.
```

R Przy wyświetlaniu obrazu *labels* trzeba w odpowiedni sposób ustawić format oraz dodać skalowanie.
Do skalowania można wykorzystać informację o liczbie znalezionych obiektów.

```
cv2.imshow("Labels", np.uint8(labels / retval * 255))
```

Następnie wyświetl prostokąt otaczający (*bounding box*), pole i indeks dla największego obiektu. Poniżej znajduje się przykładowe rozwiązanie zadania. Proszę je uruchomić i ewentualnie spróbować je zoptymalizować.

```
I_VIS = I # copy of the input image

if (stats.shape[0] > 1):   # are there any objects

  tab = stats[1:,4] # 4 columns without first element
  pi = np.argmax( tab ) # finding the index of the largest item
  pi = pi + 1 # increment because we want the index in stats, not in tab
  # drawing a bbox
  cv2.rectangle(I_VIS,(stats[pi,0],stats[pi,1]),(stats[pi,0]+stats[pi,2],stats[pi
      ,1]+stats[pi,3]),(255,0,0),2)
  # print information about the field and the number of the largest element
  cv2.putText(I_VIS,"%f" % stats[pi,4],(stats[pi,0],stats[pi,1]),cv2.
      FONT_HERSHEY_SIMPLEX,0.5,(255,0,0))
  cv2.putText(I_VIS,"%d" %pi,(np.int(centroids[pi,0]),np.int(centroids[pi,1])),cv2.
      FONT_HERSHEY_SIMPLEX,1,(255,0,0))
```

Komentarze do przykładowego rozwiązania:

- `stats.shape[0]` to liczba obiektów. Ponieważ funkcja zlicza też obiekt o indeksie 0 (tj. tło), w sprawdzeniu czy są obiekty warunek jest > 1
- kolejne dwie linie to obliczenie indeksu maksimum z kolumny numer 4 (pole).
- uzyskany indeks należy inkrementować, ponieważ w analizie pominęliśmy element 0 (tło).
- do rysowania prostokąta otaczającego na obrazie wykorzystujemy funkcję `rectangle` z OpenCV. Składnia `"%f" % stats[pi,4]` pozwala wypisać wartość w odpowiednim formacie (f - float). Kolejne parametry to współrzędne dwóch przeciwległych wierzchołków prostokąta. Następnie kolor w formacie (B, G, R), a na końcu grubość linii. Szczegóły w dokumentacji funkcji.

- do wypisywania tekstu na obrazie służy funkcja `putText`. Do określenia pozycji pola tekstowego podaje się współrzędne lewego dolnego wierzchołka. Następnie czcionka (pełna lista w dokumentacji), rozmiar i kolor.

Ćwiczenie 2.4 Wykorzystaj funkcję `cv2.connectedComponentsWithStats` do indeksacji oraz oblicz parametry otrzymanych obiektów. Wyświetl prostokąt otaczający, pole oraz indeks dla największego obiektu. Na podstawie wskazówek i przykładów zamieszczonych w tekscie powyżej spróbuj zoptymalizować rozwiązanie. ■

2.5 Ewaluacja wyników detekcji obiektów pierwszoplanowych

Aby ocenić algorytm detekcji obiektów pierwszoplanowych, należy porównać zwracaną przez niego maskę obiektów z maską referencyjną (ang. *ground truth*). Porównywanie odbywa się na poziomie poszczególnych pikseli. Jeśli wykluczy się cienie (tak jak założyliśmy na wstępie), możliwe są cztery sytuacje:

- TP – wynik prawdziwie dodatni (ang. *true positive*) – piksel należący do obiektu z pierwszego planu jest wykrywany jako piksel należący do obiektu z pierwszego planu,
- TN – wynik prawdziwie ujemny (ang. *true negative*) – piksel należący do tła jest wykrywany jako piksel należący do tła,
- FP – wynik fałszywie dodatni (ang. *false positive*) – piksel należący do tła jest wykrywany jako piksel należący do obiektu z pierwszego planu,
- FN – wynik fałszywie ujemny (ang. *false negative*) – piksel należący do obiektu jest wykrywany jako piksel należący do tła.

Na podstawie tych wartości można policzyć szereg miar. W dalszej części wykorzystamy trzy: precyzję (ang. *precision* – P), czułość (ang. *recall* – R) oraz tzw. miarę $F1$. Zdefiniowane są one następująco:

$$P = \frac{TP}{TP+FP} \tag{2.1}$$

$$R = \frac{TP}{TP+FN} \tag{2.2}$$

$$F1 = \frac{2PR}{P+R} \tag{2.3}$$

Miara F1 jest z zakresu [0;1], przy czym im jej wartość jest większa, tym lepiej.

Ćwiczenie 2.5 Zaimplementuj obliczanie miar P, R i $F1$.

R Przy czym obliczenia wykonujemy tylko wtedy, gdy dostępna jest poprawna mapa referencyjna. W tym celu należy sprawdzić zależność licznika ramki i wartości z pliku `temporalROI.txt` – musi się on zawierać w zakresie tam opisanym.

■

Ćwiczenie 2.6 Podsumowując, należy wykonać detekcję obiektów ruchomych:

1. Wczytaj sekwencję `pedestrian`, a następnie `highway` i `office`.
2. Dokonaj detekcji zmian: odejmowanie klatek i binaryzacja.
3. Usuń szum (np. filtrem medianowym lub Gaussa).
4. Zastosuj operacje morfologiczne.
5. Wykonaj indeksację.
6. Przeprowadź ewaluację.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia, po wykonaniu zadania zgłoś swoją gotowość prowadzącemu zajęcia. Równocześnie możesz zgłosić swoją gotowość po wykonaniu wszystkich zadań dotyczących poruszanego tematu zajęć. Po akceptacji z jego strony, umieść skrypt o rozszerzeniu `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL.

■

R Informacje dotyczące kolejnych zajęć.

1. W ramach dalszych ćwiczeń będziemy poznawać kolejne algorytmy i funkcjonalności języka Python. Jednak nie kładziemy szczególnej uwagi na poznawanie języka Python, ale na wykorzystaniu w praktyce kolejnych algorytmów pojawiających się na wykładach. Przedmiot Zaawansowane Algorytmy Wizyjne nie jest kursem Pythona!
2. Przedstawione rozwiązania należy zawsze traktować jako przykładowe. Na pewno problem można rozwiązać inaczej, a czasem lepiej.

2.6 Przykładowe rezultaty algorytmu do detekcji ruchomych obiektów pierwszoplanowych

W celu lepszej weryfikacji poszczególnych etapów zaproponowanej metody do detekcji ruchomych obiektów pierwszoplanowych przedstawiony zostanie przykładowy wynik dla obrazu o indeksie 350.

Rysunek 2.1: Przykładowy wynik poszczególnych etapów algorytmu. Od lewej obraz wejściowy z prostokątem otaczającym, obraz po binaryzacji, obraz po zastosowaniu filtracji medianowej oraz operacji morfologicznych (erode, dilate), obraz reprezentujący etykiety po indeksacji, obraz referencyjny.

3 Laboratorium 3

3 Segmentacja obiektów pierwszoplanowych 25
3.1 Cel zajęć
3.2 Segmentacja obiektów pierwszoplanowych
3.3 Metody oparte o bufor próbek
3.4 Aproksymacja średniej i mediany (tzw. sigma-delta)
3.5 Polityka aktualizacji
3.6 OpenCV – GMM/MOG
3.7 OpenCV – KNN
3.8 Przykładowe rezultaty algorytmu do segmentacji obiektów pierwszoplanowych
3.9 Wykorzystanie sieci neuronowej do segmentacji obiektów pierwszoplanowych

3. Segmentacja obiektów pierwszoplanowych

3.1 Cel zajęć

- zapoznanie się z zagadnieniem segmentacji obiektów pierwszoplanowych oraz problemami z tym związanymi,
- implementacja prostych algorytmów modelowania tła opartych na buforze próbek – analiza ich wad i zalet,
- implementacja algorytmów średniej bieżącej oraz aproksymacji medianowej – analiza ich wad i zalet,
- poznanie metod dostępnych w *OpenCV* – modelu Gaussian Mixture Model (zwanego też Mixture of Gaussians) oraz metody opartej na algorytmie KNN (K-Nearest Neighbours, k-najbliższych sąsiadów),
- zapoznanie się z architekturą sieci neuronowej oraz przykładową aplikacją.

3.2 Segmentacja obiektów pierwszoplanowych

Segmentacja obiektów pierwszoplanowych (ang. *foreground object segmentation*)
Segmentacja polega na wydzieleniu z obrazu interesujących obiektów. Nie musi to oznaczać *klasyfikacji* (np. *samochód*, *pieszy*); często wystarcza informacja, *gdzie* na obrazie znajduje się obiekt. Pojęcie *obiektu pierwszoplanowego* jest zależne od aplikacji (np. w monitoringu: ludzie i pojazdy).

Model tła
W najprostszym ujęciu jest to obraz „pustej sceny”, tj. bez obiektów pierwszoplanowych. W praktyce, w wielu metodach model tła jest niejawny (nie jest pojedynczym obrazem) i nie da się go wprost wyświetlić (np. GMM, ViBE, PBAS).

Inicjalizacja modelu tła
Model trzeba zainicjalizować przy starcie algorytmu. Najczęściej jako start przyjmuje się

pierwszą ramkę (lub pierwsze N ramek w metodach buforowych). Założenie: w fazie inicjalizacji nie ma obiektów pierwszoplanowych. Jeśli obiekty są obecne, model startuje z błędem, co w literaturze określa się jako *bootstrap*. Bardziej odporne podejścia analizują zmiany w czasie (i czasem także spójność przestrzenną), zakładając, że tło jest widoczne przez większość czasu.

Modelowanie (generacja) tła
Statyczny „obraz tła" działa tylko w ściśle kontrolowanych warunkach. W typowych scenach tło zmienia się (oświetlenie, pora dnia) albo przestawiane są elementy sceny (np. krzesło). Dlatego model tła powinien się adaptować; proces tej aktualizacji nazywa się modelowaniem/generacją tła. Brak adaptacji (albo zbyt wolna adaptacja) prowadzi do fałszywych detekcji, m.in. *ghost* (*duch*).

Pułapki przy modelowaniu tła
Nie istnieje jedna metoda dobra dla wszystkich scen. Typowe sytuacje problematyczne:

- szum i artefakty kompresji (MJPEG, H.264/265, MPEG),
- drżenie kamery,
- automatyka kamery (balans bieli, ekspozycja),
- zmiany oświetlenia: powolne (pora dnia) i nagłe,
- obiekty obecne podczas inicjalizacji,
- ruchome elementy tła,
- kamuflaż (podobieństwo obiektu do tła),
- obiekty przestawione w tle.

***Ghost* (duchy)**
Przykład: samochód zatrzymuje się na parkingu i po pewnym czasie zostaje włączony do tła. Gdy odjedzie, „puste miejsce" bywa wykrywane jako obiekt — to właśnie *ghost* (obiekt obecny w modelu, a nie w bieżącej ramce).

Polityka aktualizacji
Aktualizacja modelu może być:

- **konserwatywna** — uaktualniane są tylko piksele sklasyfikowane jako tło,
- **liberalna** — uaktualniane są wszystkie piksele.

Polityka konserwatywna może „zamrozić" błąd (piksele nigdy nie są poprawiane), a liberalna sprzyja wtapianiu obiektów w tło i powstawaniu *ghostów* lub smug.

Cienie
Cień rzucany przez obiekt zwykle spełnia kryteria „pierwszego planu", ale z punktu widzenia dalszej analizy jest zakłóceniem (zmienia kształt, łączy obiekty). Dlatego cień często traktuje się jako osobną klasę: nie tło i nie obiekt pierwszoplanowy.

Przykład
Na rysunku 3.1 pokazano segmentację z wykorzystaniem modelu tła. Zwróć uwagę na cień pod nogami osoby po lewej oraz osobę za barierką (słabo widoczną w bieżącej ramce).

3.3 Metody oparte o bufor próbek

W ramach ćwiczenia zaprezentowane zostaną dwie metody: średnia z bufora oraz mediana z bufora. Rozmiar bufora przyjmiemy na $N = 60$ próbek. W każdej iteracji algorytmu należy usunąć ostatnią ramkę z bufora, dodać bieżącą (bufor działa na zasadzie kolejki FIFO) oraz obliczyć średnią lub medianę z bufora.

a) b) c)

Rysunek 3.1: Przykład segmentacji obiektów pierwszoplanowych z wykorzystaniem modelowania tła. a) bieżąca ramka, b) model tła, c) maska obiektów pierwszoplanowych. Źródło: sekwencja PETS 2006.

1. Na początku należy zadeklarować bufor o rozmiarze $N \times YY \times XX$ (polecenie `np.zeros`), gdzie YY i XX to odpowiednio wysokość i szerokość ramki z sekwencji *pedestrians*.

```
BUF = np.zeros((YY,XX,N),np.uint8)
```

   Należy też zainicjalizować licznik bufora (np. `iN`) wartością 0. Parametr `iN` pełni rolę wskaźnika: wskazuje pozycję w buforze, w której należy usunąć najstarszą ramkę i zastąpić ją ramką bieżącą.
2. Obsługa bufora powinna wyglądać następująco: w pętli na pozycji wskazywanej przez `iN` należy zapisać bieżącą ramkę w skali szarości.

```
BUF[:,:,iN] = IG;
```

   Następnie licznik `iN` należy inkrementować i sprawdzać, czy nie osiągnął rozmiaru bufora (N). Jeśli tak, to trzeba go ustawić na 0.
3. Obliczenie średniej lub mediany realizuje się za pomocą funkcji `mean` lub `median` z biblioteki *numpy*. Jako parametr podaje się wymiar (oś), dla którego ma być liczona wartość (pamiętać o indeksowaniu od zera). Aby wszystko działało poprawnie, należy dokonać konwersji wyniku na `uint8`.
4. Ostatecznie realizujemy odejmowanie tła, tzn. od bieżącej sceny odejmujemy model, a wynik binaryzujemy – analogicznie jak przy różnicy sąsiednich ramek. Dodatkowo wykorzystujemy filtrację medianową maski obiektów i/lub operacje morfologiczne.
5. Wykorzystując stworzony w ramach poprzedniego ćwiczenia kod, porównaj działanie metody ze średnią i medianą. Zanotuj wartości wskaźnika $F1$ dla obu przypadków.

R Mediana może być obliczana przez dłuższą chwilę.

Zastanów się, dlaczego mediana działa lepiej. Sprawdź działanie na innych sekwencjach – w szczególności dla sekwencji *office*. Zaobserwuj zjawisko smużenia oraz wtapiania się sylwetki w tło, a także powstawania *ghostów*.

Ćwiczenie 3.1 Zaimplementuj algorytm (średniej i mediany) na podstawie powyższego opisu. ■

3.4 Aproksymacja średniej i mediany (tzw. sigma-delta)

Użycie bufora próbek wymaga znacznych zasobów pamięci. Średnią można aktualizować w prosty sposób (warto zastanowić się, jak zaktualizować średnią w buforze bez ponownego przeliczania wszystkich elementów). W przypadku mediany sytuacja jest trudniejsza: nie istnieją równie szybkie algorytmy jej wyznaczania, choć nie trzeba sortować wszystkich elementów w każdej iteracji. Dlatego częściej stosuje się metody, które nie wymagają bufora. W przypadku aproksymacji średniej wykorzystuje się zależność:

$$BG_N = \alpha I_N + (1-\alpha)\, BG_{N-1} \tag{3.1}$$

gdzie: I_N – bieżąca ramka, BG_N – model tła, α – parametr wagowy (zwykle 0,01–0,05, choć wartość zależy od konkretnej aplikacji).

Aproksymację mediany uzyskuje się, wykorzystując zależność:

$$BG_N = \begin{cases} BG_{N-1}+1, & \text{gdy } BG_{N-1} < I_N, \\ BG_{N-1}-1, & \text{gdy } BG_{N-1} > I_N, \\ BG_{N-1}, & \text{w przeciwnym razie.} \end{cases} \tag{3.2}$$

Ćwiczenie 3.2 Zaimplementuj obie metody.

1. Jako pierwszy model tła przyjmij pierwszą ramkę z sekwencji. Zanotuj wartość wskaźnika $F1$ dla tych dwóch metod (parametr α ustal na 0.01). Zaobserwuj czas działania.
2. Poeksperymentuj z wartością parametru α. Zobacz jaki ona ma wpływ na model tła.

■

R Dla metody średniej bieżącej kluczowe jest, aby **model tła był przechowywany w formacie zmiennoprzecinkowym** (*float64*).
We wzorze 3.1 uniknięcie stosowania pętli po całym obrazie jest oczywiste. Dla zależności 3.2 również jest to możliwe, ale wymaga chwili zastanowienia. Warto w tym celu sięgać do dokumentacji *numpy*. Proszę też zauważyć, że w Pythonie można wykonywać działania arytmetyczne na wartościach boolowskich.

3.5 Polityka aktualizacji

Jak dotąd stosowaliśmy liberalną politykę aktualizacji – aktualizowaliśmy wszystko. Spróbujemy teraz wykorzystać podejście konserwatywne.

Ćwiczenie 3.3 Polityka aktualizacji.

1. Dla wybranej metody zaimplementuj konserwatywne podejście do aktualizacji. Sprawdź jego działanie.

   R Wystarczy zapamiętać poprzednią maskę obiektów i odpowiednio wykorzystać ją w procedurze aktualizacji.

2. Zwróć uwagę na wartość wskaźnika F1 oraz na model tła. Czy pojawiły się w nim jakieś błędy ?

■

3.6 OpenCV – GMM/MOG

W bibliotece OpenCV dostępna jest jedna z najpopularniejszych metod segmentacji obiektów pierwszoplanowych: Gaussian Mixture Models (GMM), nazywana też Mixture of Gaussians (MoG) – obie nazwy występują równolegle w literaturze. W największym skrócie scena jest modelowana za pomocą kilku rozkładów Gaussa (np. średniej jasności/koloru oraz odchylenia standardowego). Każdy rozkład opisany jest również wagą, która odzwierciedla, jak często był on obserwowany na scenie (tj. prawdopodobieństwo wystąpienia). Rozkłady o największych wagach stanowią tło. Segmentacja polega na obliczaniu odległości pomiędzy bieżącą obserwacją piksela a każdym z rozkładów. Jeśli piksel jest podobny do rozkładu uznanego za tło, klasyfikowany jest jako tło; w przeciwnym przypadku zostaje uznany za obiekt. Model tła jest również aktualizowany w sposób zbliżony do równania 3.1. Zainteresowane osoby odsyłamy do literatury, np. do pracy Stauffera i Grimsona.

Ćwiczenie 3.4 Metoda ta jest przykładem algorytmu wielowariantowego – model tła ma kilka możliwych reprezentacji (wariantów).

1. Wykorzystaj klasę `BackgroundSubtractorMOG2`, tworząc obiekt za pomocą `create BackgroundSubtractorMOG2`. W pętli, dla każdego obrazu, wywołuj metodę `apply()`.
2. Poeksperymentuj z parametrami *history* i *varThreshold* oraz wyłącz detekcję cieni. W metodzie `apply()` można również ustalić współczynnik uczenia – *learningRate*.
3. Wyznacz miarę $F1$ (dla metody bez detekcji cieni). Zaobserwuj, jak działa metoda. Zwróć uwagę, że nie da się „wyświetlić” modelu tła.

R W pakiecie OpenCV dostępna jest wersja GMM nieco inna niż oryginalna – między innymi wyposażona w moduł detekcji cieni oraz dynamicznie zmienianą liczbę rozkładów Gaussa.

■

3.7 OpenCV – KNN

Druga z metod dostępnych w OpenCV to KNN (BackgroundSubtractorKNN).

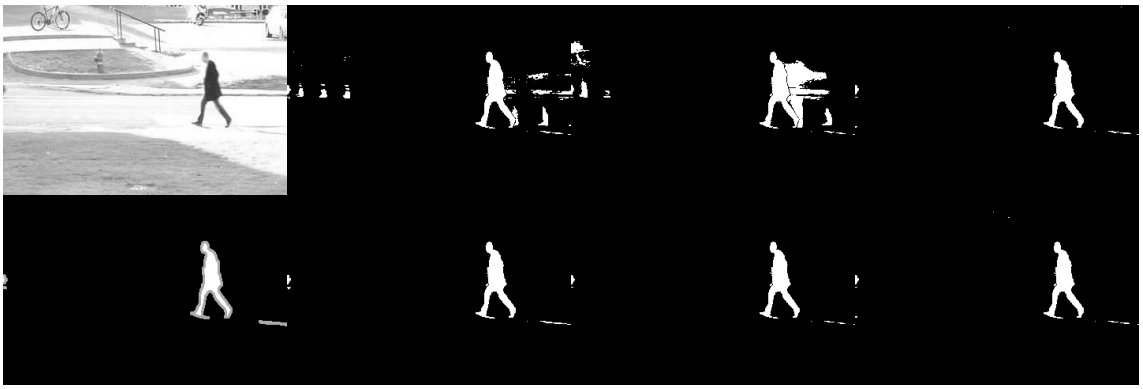
Ćwiczenie 3.5 Proszę uruchomić metodę KNN i dokonać jej ewaluacji. ■

3.8 Przykładowe rezultaty algorytmu do segmentacji obiektów pierwszoplanowych

W celu lepszej weryfikacji poszczególnych etapów zaproponowanych metod do segmentacji obiektów pierwszoplanowych przedstawione zostaną przykładowe wyniki dla wybranych ramek obrazu.

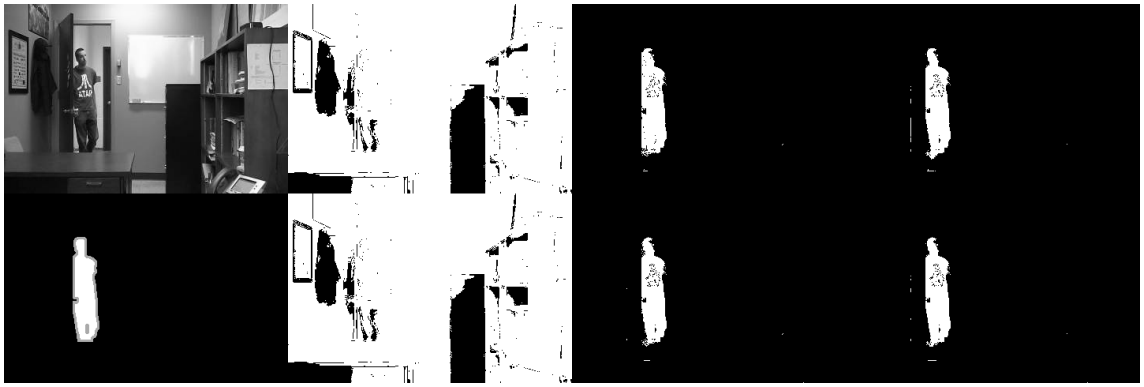
Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia, po wykonaniu zadania zgłoś swoją gotowość prowadzącemu zajęcia. Równocześnie możesz zgłosić swoją gotowość po wykonaniu wszystkich zadań dotyczących poruszanego tematu zajęć. Po akceptacji z jego strony, umieść skrypt o rozszerzeniu `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL.



Rysunek 3.2: Przykładowy wynik poszczególnych wersji algorytmu do segmentacji obiektów pierwszoplanowych dla zbioru *pedestrian*. Indeks ramki to 600. Pierwsza od lewej kolumna to obraz wejściowy wraz z obrazem referencyjnym. Pierwszy wiersz dotyczy algorytmów wykorzystujących średnią, drugi wiersz wykorzystuje medianę. Kolejno od drugiej kolumny prezentowany jest wynik algorytmu: liberalna polityka aktualizacji, następnie aproksymacja funkcji z liberalną polityką aktualizacji oraz aproksymacja funkcji z konserwatywną polityką aktualizacji.

Rysunek 3.3: Przykładowy wynik poszczególnych wersji algorytmu do segmentacji obiektów pierwszoplanowych dla zbioru *highway*. Indeks ramki to 1200. Pierwsza od lewej kolumna to obraz wejściowy wraz z obrazem referencyjnym. Pierwszy wiersz dotyczy algorytmów wykorzystujących średnią, drugi wiersz wykorzystuje medianę. Kolejno od drugiej kolumny prezentowany jest wynik algorytmu: liberalna polityka aktualizacji, następnie aproksymacja funkcji z liberalną polityką aktualizacji oraz aproksymacja funkcji z konserwatywną polityką aktualizacji.



Rysunek 3.4: Przykładowy wynik poszczególnych wersji algorytmu do segmentacji obiektów pierwszoplanowych dla zbioru *office*. Indeks ramki to 600. Pierwsza od lewej kolumna to obraz wejściowy wraz z obrazem referencyjnym. Pierwszy wiersz dotyczy algorytmów wykorzystujących średnią, drugi wiersz wykorzystuje medianę. Kolejno od drugiej kolumny prezentowany jest wynik algorytmu: liberalna polityka aktualizacji, następnie aproksymacja funkcji z liberalną polityką aktualizacji oraz aproksymacja funkcji z konserwatywną polityką aktualizacji.

3.9 Wykorzystanie sieci neuronowej do segmentacji obiektów pierwszoplanowych

Do segmentacji obiektów pierwszoplanowych można wykorzystać również metody oparte na architekturze sieci neuronowych. Przykładem takiego rozwiązania jest praca[1]. Zaproponowane podejście wykorzystuje konwolucyjną sieć neuronową BSUV–Net (ang. Background Subtraction of Unseen Videos Net). Architektura bazuje na strukturze U–Net z połączeniami resztkowymi (ang. residual connections) i składa się z enkodera oraz dekodera. Wejściem jest konkatenacja kilku obrazów, tj. bieżącej ramki oraz dwóch ramek tła w różnych odstępach czasowych, wraz z odpowiadającymi im semantycznymi mapami segmentacji. Wyjściem jest maska zawierająca wyłącznie obiekty pierwszoplanowe.

Szczegóły dotyczące tej sieci neuronowej przedstawiono w artykule: arXiv:1907.11371. Kod wraz z przykładowymi danymi oraz wytrenowanymi modelami udostępniono w serwisie GitHub w repozytorium BSUV-Net-inference.

Ćwiczenie 3.6 **Zadanie nieobowiązkowe.** Uruchom konwolucyjna sieć neuronową BSUV-Net.

- Ściągnij z platformy UPeL wszystkie pliki do sieci neuronowej.
- Wykorzystana zostanie baza danych `pedestrians`, ale odpowiednio przeskalowana do rozmiaru wejścia sieci oraz przekonwertowana na sekwencje wideo,
- Otwórz plik `infer_config_manualBG.py` oraz zmodyfikuj ścieżki do:
  - w klasie `SemanticSegmentation` podaj ścieżkę absolutną do folderu segmentation – zmienna `root_path`,
  - w klasie `BSUVNet` podaj ścieżkę do modelu sieci `BSUV-Net-2.0.mdl` w folderze `trained_models` – `model_path`,
  - w klasie `BSUVNet` podaj ścieżkę do obrazu, który zawiera jedynie tło – pierwsze zdjęcie ze zbioru `pedestrians` – zmienna `empty_bg_path`
- W pliku `inference.py` podaj ścieżkę do wejścia sieci, czyli sekwencja wideo `pedestrians` – zmienna `inp_path` oraz ścięźkę do miejsca gdzie ma być zapisane wyjście sieci wraz z nazwą pliku – zmienna `out_path`.

■

[1] Tezcan, Ozan and Ishwar, Prakash and Konrad, Janusz; BSUV-Net: A Fully-Convolutional Neural Network for Background Subtraction of Unseen Videos, arXiv:1907.11371

4 Extra laboratorium 1

4 Uogólniona transformata Hougha 35
4.1 Cel zajęć
4.2 Implementacja uogólnionej transformaty Hougha

4. Uogólniona transformata Hougha

4.1 Cel zajęć

- implementacja uogólnionej transformaty Hougha,
- wyszukiwanie wzorców za pomocą uogólnionej transformaty Hougha.

4.1.1 R-table

4.2 Implementacja uogólnionej transformaty Hougha

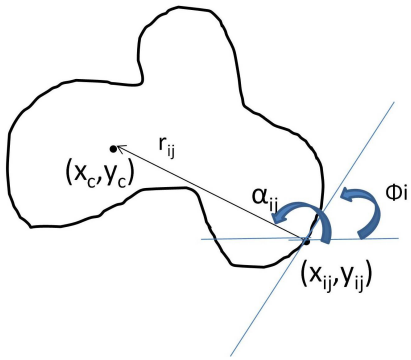
Ćwiczenie 4.1 Wyszukiwanie wzorców.

1. Ze strony kursu pobierz archiwum z danymi do ćwiczenia i rozpakuj je we własnym katalogu roboczym.
2. Utwórz nowy skrypt. Na podstawie obrazu ze wzorcem *trybik.jpg* stworzymy tablicę *R-table*. W tym celu wyznacz **kontury** oraz **gradienty** na obrazie wzorca.
3. Aby **wyznaczyć kontur** należy najpierw przeprowadzić konwersję obrazu na odcień szarości oraz binaryzację z odpowiednim progiem. Następnie wykorzystaj funkcję `cv2.findContours` do uzyskania konturów występujących na obrazie.

R Zaneguj obraz – `cv2.bitwise_not` – przed przesłaniem go do funkcji, gdyż zawiera on czarny kontur na białym tle, a funkcja `findContours` oczekuje odwrotnego ustawienia

i	ϕ	R_{ϕ_i}
1	0	$(r_{11}, \alpha_{11})(r_{12}, \alpha_{12}) \dots (r_{1n}, \alpha_{1n})$
2	$\Delta\phi$	$(r_{21}, \alpha_{22})(r_{22}, \alpha_{22}) \dots (r_{2m}, \alpha_{2m})$
3	$2\Delta\phi$	$(r_{31}, \alpha_{31})(r_{32}, \alpha_{32}) \dots (r_{3k}, \alpha_{3k})$
...	...	...

Tabela 4.1: Budowa R-table.



Rysunek 4.1: Budowa R-table. Źródło: Generalised Hough transform - Wikipedia

Zwróć uwagę, aby uzyskać listy wszystkich punktów konturu – zastosuj parametr `CHAIN_APPROX_NONE`.

R W przypadku, gdy kontur zostanie podzielony można wykorzystać operacje morfologiczne, aby zapobiec podziałom lub ustawić parametr `RETR_TREE`, który pozwala ułożyć kontury w kolejności od najdłuższego do najkrótszego i wybrać najdłuższy kontur.

4. Wyświetl kontur za pomocą funkcji np. `cv2.drawContours`.
5. Wylicz gradienty, wykorzystując filtry **Sobela**.
   - Oblicz amplitudę gradientu,
   - Oblicz orientację gradientu.

   R Wartość amplitudy gradientu warto znormalizować przez jej wartość maksymalną.

6. **Wybierz punkt referencyjny** – będzie to środek ciężkości wzorca wyznaczany ze zbinaryzowanego obrazu wzorca z wykorzystaniem momentów. Do wyznaczenia środka ciężkości można wykorzystać momenty centralne *m00*, *m10* i *m01* (funkcja `cv2.moments(bin, 1)`).
7. Do wypełnienia `R-table` będą potrzebne wektory łączące punkty konturu/konturów z punktem referencyjnym. Do `R-table` wpisujemy **długości tych wektorów** oraz **kąty** jakie tworzą z osią OX. Miejsce wpisania do tablicy `R-table` wyznacza orientacja gradientu w punkcie konturu (wyznaczona na podstawie filtracji maskami Sobela), **przy czym proszę przeliczyć radiany na stopnie** – `R-table` będzie miała 360 wierszy. Stosujemy dokładność wynoszącą 1 stopień.
   Wówczas np. *Rtable[30]* będzie listą współrzędnych biegunowych punktów konturu, których orientacja wyliczona na podstawie gradientów wynosi około 30°.
8. Na podstawie obrazu `trybiki2.jpg` oraz `R-table` z poprzedniego punktu wypełnij dwuwymiarową przestrzeń Hougha – wylicz ponownie gradient w każdym punkcie i dla punktów, których znormalizowana wartość gradientu przekracza 0.5, określ orientację w tym punkcie, a następnie zwiększ wartość o 1 w przestrzeni akumulacyjnej w punktach zapisanych w `R-table` według zależności:

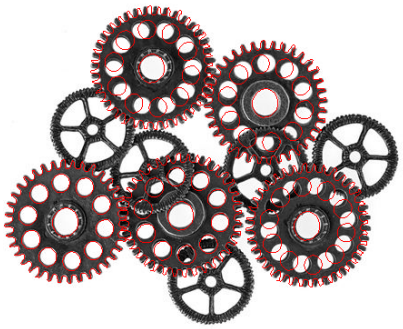
```
x1 = -r*np.cos(fi) + x
y1 = -r*np.sin(fi) + y
```

gdzie r,fi – wartości z odpowiedniego wiersza w *R-table*, x,y – współrzędne punktu, dla którego gradient przekracza 0.5.
Warto też sobie wyświetlić postać przestrzeni Hougha.

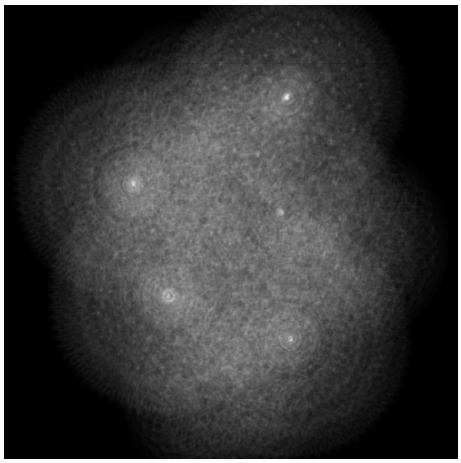
9. Wyszukaj maksimum w przestrzeni Hougha i zaznacz je na obrazie wejściowym – `trybiki2.jpg`.
10. Wynikiem działania powyższego algorytmu jest zaznaczenie znalezionego maksimum na obrazie oraz nałożenie konturu wzorca wokół tego punktu. Wyznacz wszystkie pięć maksimów na obrazie.

    R Aby bez problemu znaleźć kolejne maksima, warto wcześniej wyzerować pewien obszar wokół już znalezionego maksimum.

11. Przykładowe rozwiązanie zostało przedstawione na rys. 4.2, dodatkowo na rys. 4.3 zilustrowano przestrzeń Hougha.



Rysunek 4.2: Przykładowe rozwiązanie.



Rysunek 4.3: Przestrzeń Hougha.

5 Mini projekt 1

5 Projekt 1 41
5.1 Cel projektu
5.2 Przykładowa literatura

5. Projekt 1

Temat 1: Wykrywanie wad włosia szczoteczki

Wykrywanie wad włosia szczoteczki (ang. *toothbrush bristle defect detection*) realizuje się głównie za pomocą systemów wizyjnych i uczenia maszynowego. Dane do tego typu zadań obejmują obrazy wysokiej rozdzielczości oraz zdjęcia z mikroskopów elektronowych (SEM), co pozwala na analizę jakości końcówek włosia.

W systemach produkcyjnych (ang. *quality control*) najczęściej analizuje się:

- **Włosie rozszczepione/rozłożone** (ang. *bristle splaying*) – deformacja włosia.
- **Ścieranie** (ang. *abrasion*) – zniszczenie końcówek włosia.
- **Błędy w stapianiu/mocowaniu** (ang. *bristle stapling defects*) – nieprawidłowe osadzenie pęczków.
- **Niewłaściwe zaokrąglenie końcówek** (ang. *irregular bristle end rounding*).
- **Braki we włosiu** (ang. *hair loss/missing bristles*).

Temat 2: Wykrywanie wad w kablu elektrycznym

Wykrywanie wad w kablu elektrycznym (ang. *electric cable defect detection*) polega na identyfikacji defektów izolacji oraz nieprawidłowości budowy wewnętrznej kabla na podstawie obrazów. W przedstawionym wariancie dane mogą stanowić zdjęcia **przekrojów poprzecznych** (kabel przecięty i sfotografowany), co pozwala ocenić położenie żył względem siebie, poprawność ułożenia oraz stan przewodników i izolacji.

W systemach produkcyjnych (ang. *quality control*) oraz w analizie przekrojów najczęściej wykrywa się:

- **Brak jednej żyły/przewodu** (ang. *missing conductor/core*) – niekompletna wiązka w kablu wielożyłowym.
- **Zagięty/nagiety przewód** (ang. *bent conductor*) – nienaturalne wygięcie żyły widoczne w przekroju.
- **Przemieszczenie żyły, mimośrodowość** (ang. *conductor displacement, eccentricity*) – żyła nie jest osiowo ułożona w izolacji.
- **Uszkodzenia żyły** (ang. *strand breakage / conductor damage*) – przerwane druciki, deformacje przekroju.

- **Zwarcie/mostek materiałowy między żyłami** (ang. *short / bridging*) – kontakt przewodników lub brak separacji.
- **Pęcherze, puste przestrzenie** (ang. *voids and bubbles*) – ubytki w izolacji lub wypełnieniu.
- **Nierównomierna grubość izolacji** (ang. *insulation thickness variation*) – lokalne osłabienie dielektryczne.
- **Nacięcia i pęknięcia izolacji** (ang. *cuts and cracks*) – rozdarcia powłoki widoczne na przekroju.

Temat 3: Wykrywanie wad tranzystora

Wykrywanie wad tranzystora (ang. *transistor defect detection*) dotyczy kontroli jakości elementów półprzewodnikowych na podstawie obrazów (np. zdjęć obudowy, wyprowadzeń oraz – w zależności od zadania – mikrofotografii struktury krzemowej). Celem jest wykrycie defektów mechanicznych i montażowych, które mogą prowadzić do awarii lub pogorszenia parametrów.

W systemach produkcyjnych (ang. *quality control*) najczęściej analizuje się:

- **Zagięte/skręcone wyprowadzenia** (ang. *bent/twisted leads*) – utrudniony montaż i ryzyko zwarć.
- **Brakujące lub uszkodzone wyprowadzenie** (ang. *missing/broken lead*).
- **Utlenienie/korozja** (ang. *oxidation/corrosion*) na wyprowadzeniach.
- **Pęknięcia i wyszczerbienia obudowy** (ang. *package cracks/chips*) – ryzyko nieszczelności i uszkodzeń mechanicznych.
- **Wady znakowania** (ang. *marking defects*) – nieczytelny nadruk, przesunięcie lub brak oznaczeń.
- **Zanieczyszczenia i pozostałości procesu** (ang. *contamination/residue*) – pył, przebarwienia, pozostałości topnika.
- **Odchyłki wymiarowe/geometrii** (ang. *dimensional/geometric defects*) – np. nieprawidłowy rozstaw pinów.

5.1 Cel projektu

Celem projektu jest opracowanie systemu wizyjnego umożliwiającego automatyczne wykrywanie wad wybranych obiektów. W ramach projektu należy:

- przygotować dane (obrazy) oraz zdefiniować klasy wad,
- zaproponować i zaimplementować etap wstępnego przetwarzania (np. normalizacja, odszumianie, poprawa kontrastu),
- wyznaczyć obszary istotne do analizy (np. segmentacja końcówek włosia) lub cechy opisujące defekty,
- dobrać i wytrenować model klasyfikacji/wykrywania (lub zbudować regułowy detektor bazujący na cechach),
- ocenić jakość rozwiązania (np. accuracy, precision/recall, macierz pomyłek) oraz omówić ograniczenia.

Zbiór danych dostępny jest pod adresem: https://drive.google.com/drive/folders/1nXdqY60uZIEWWwLzuxtzRkXygI2xNbS6?usp=sharing.

5.2 Przykładowa literatura

- Bao, Nengsheng, et al. *A deep learning-based process monitoring system for toothbrush manufacturing defect characterization. Procedia CIRP* **118** (2023): 1072–1077.

- Bergmann, Paul, et al. *MVTec AD–A comprehensive real-world dataset for unsupervised anomaly detection.* In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).